

PYTHON 을 이용한 데이터 분석 · 14 강

차원 축소

PCA (주성분분석 , Principal Component Analysis)

이론편 — 특성이 많아질수록 생기는 문제, 그리고 압축의 기술



오늘의 학습 목표



차원의 저주 이해하기

특성이 많아질수록 계산 · 시각화 · 과적합 측면에서 어떤 문제가 생기는지



PCA의 핵심 아이디어

정보 손실을 최소화하며 더 적은 수의 새로운 축 (주성분) 으로 데이터를 압축하는 원리



표준화의 중요성

PCA를 적용하기 전에 왜 반드시 스케일을 맞춰야 하는지



설명된 분산 비율과 `n_components` 선택

스크리 플롯과 누적 분산으로 주성분 개수를 정하는 방법



PCA의 실전 트레이드오프

압축이 항상 정확도 · 속도를 개선하지는 않는다는 것을 실제 데이터로 확인

지난 시간 복습 — 특성이 26 개 ?

13 강 기억나시나요 ? 성인 소득 데이터를 `get_dummies()` 로 원 - 핫 인코딩하니 원래 9 개였던 컬럼이 26 개 특성으로 늘어났습니다 . 특성이 이렇게 많아지면 어떤 문제가 생길까요 ?

계산량 폭발

특성이 늘어날수록 학습 · 예측에 필요한 연산량과 메모리가 증가합니다 .

시각화 불가능

사람은 3 차원까지만 눈으로 볼 수 있습니다 . 26 차원 데이터를 어떻게 그림으로 그릴까요 ?

과적합 위험 증가

특성이 샘플 수보다 지나치게 많아지면 모델이 노이즈까지 암기하기 쉬워집니다 .

중복 · 상관 정보

여러 특성이 비슷한 정보를 담고 있다면 (다중공선성), 굳이 다 가지고 있을 필요가 있을까요 ?

CONCEPT

PCA란 무엇인가 ?



정보를 최대한 보존하면서, 더 적은 수의 새로운 축으로 데이터를 압축하는 방법

비유로 이해하기 사진을 압축 (JPEG) 할 때 화질을 거의 유지하면서 파일 크기를 줄이듯이, PCA 는 데이터의 '핵심 패턴 (분산)' 은 최대한 유지하면서 특성 개수를 줄입니다 . 원래 특성을 몇 개 골라서 버리는 것이 아니라, 원래 특성들을 조합한 완전히 새로운 축을 만든다는 점이 핵심입니다 .

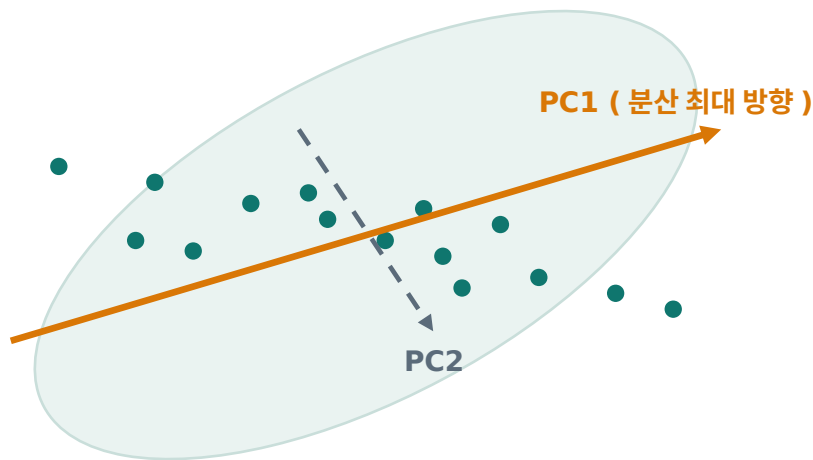
PCA(4 개 특성) → 2 개의 새로운 축 (주성분 , Principal Component) 으로 압축

sepal length, sepal width, petal length, petal width (4 차원) → PC1, PC2 (2 차원)

CONCEPT

분산과 주성분 — 가장 넓게 퍼진 방향 찾기

PCA 는 데이터가 가장 넓게 (다양하게) 퍼져 있는 방향을 첫 번째 축 (PC1) 으로 삼습니다 .



PC1, PC2 는 서로 직각 (수직)

첫 번째 주성분 (PC1) 이 잡아내지 못한 분산 중 , 가장 큰 방향이 두 번째 주성분 (PC2) 이 됩니다 . 이런 식으로 원래 특성 개수만큼 주성분을 만들 수 있습니다 .

💡 분산이 크다 = 정보가 많다

데이터가 넓게 퍼져 있을수록 그 방향은 ' 개체를 구별하는 데 ' 유용한 정보를 담고 있다고 봅니다 .

PCA 작동 원리 — 4 단계로 이해하기

1

표준화 (Standardization)

모든 특성의 스케일을 평균 0, 표준편차 1로 맞춥니다.

2

공분산 행렬 계산

특성들이 서로 얼마나 함께 변하는지 (상관관계)를 계산합니다.

3

고유벡터 · 고유값 계산

분산이 가장 큰 방향 (고유벡터 = 주성분 방향)과 그 크기 (고유값 = 중요도)를 구합니다.

4

새 축으로 투영 (Projection)

원래 데이터를 중요도 순으로 정렬된 새로운 축 위에 다시 표현합니다.



WHY STANDARDIZE

표준화가 왜 필수일까 ?

PCA 는 ' 분산이 큰 방향 ' 을 중요하다고 판단합니다 . 그런데 특성마다 단위가 다르면 어떻게 될까요 ?

```
income_df[['age', 'capital_gain']].std()
# age          약 13    (20~65 세 범위 )
# capital_gain  약 8,000  (0~ 수만 달러 범위 )
```

표준화를 안 하면 ? capital_gain 의 표준편차 (약 8,000) 가 age (약 13) 보다 훨씬 크기 때문에 , PCA 는 capital_gain 만 ' 중요한 정보 ' 로 착각하고 PC1 을 온통 capital_gain 방향으로 정해버립니다 . 실제로는 두 특성 모두 의미가 있는데도 말이죠 .

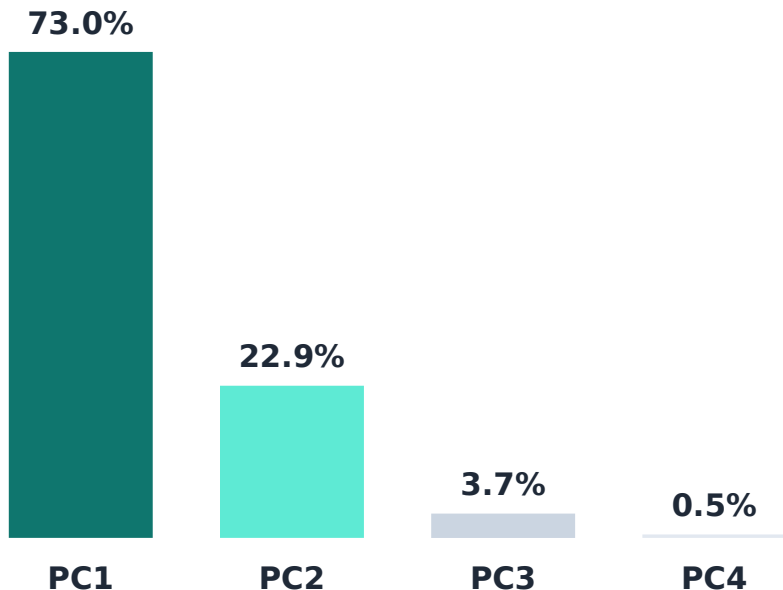
→ 그래서 PCA 전에는 항상 StandardScaler 로 모든 특성을 평균 0, 표준편차 1 로 맞추습니다 .

CONCEPT

설명된 분산 비율 (Explained Variance Ratio)



각 주성분이 원본 데이터의 분산 (정보) 을 얼마나 ' 설명 ' 하는지 나타내는 비율입니다 .



Iris 데이터 예시

PC1 이 72.96%, PC2 가 22.85% 의 분산을 설명합니다 .

$$\text{PC1} + \text{PC2} = 95.81\%$$

→ 4 개 특성을 2 개로 줄여도 정보의 약 96% 를 그대로 유지한다는 뜻입니다 !

몇 개의 주성분을 써야 할까 ?



스크리 플롯 (Scree Plot)

주성분별 설명된 분산을 그래프로 그려서, 기울기가 완만해지는 '팔꿈치 (elbow)' 지점을 찾습니다.



누적 분산 기준

누적 설명된 분산이 80%, 90%, 95% 등 원하는 기준을 넘는 최소 개수를 선택합니다.



실험적으로 결정

실제로 모델 성능 (정확도 등) 이 어떻게 변하는지 비교해서 최종 결정합니다.

```
np.cumsum(pca.explained_variance_ratio_)    #    누적 분산 비율 계산
```

PCA 는 어디에 쓰일까 ?



시각화

고차원 데이터를 2~3 차원으로 압축해서 눈으로 볼 수 있게 합니다 .



노이즈 제거 · 속도 향상

분산이 작은 (노이즈에 가까운) 주성분을 제거해 계산을 가볍게 합니다 .



다중공선성 완화

서로 상관관계가 높은 특성들을 독립적인 주성분으로 재구성합니다 .



전처리 단계

다른 모델 (회귀 , 분류 , 군집화 등) 에 넣기 전 차원을 줄이는 용도로 사용합니다 .

LIMITATIONS

PCA 의 한계 — 만능은 아니다



해석이 어려움

주성분 (PC1, PC2...) 은 원래 특성들의 조합이라 , ' 이게 무슨 의미인지 ' 직관적으로 알기 어렵습니다 .



선형 관계만 포착

PCA 는 직선 (선형) 방향의 분산만 찾습니다 . 곡선 형태로 퍼진 데이터는 잘 압축하지 못합니다 .



스케일링에 민감

표준화를 빠뜨리면 결과가 완전히 달라질 수 있습니다 .



이상치에 민감

극단적인 값이 있으면 분산 계산이 왜곡되어 주성분 방향이 틀어질 수 있습니다 .

PREVIEW 1

실제 결과 미리보기 — Iris (4→2 차원)



붓꽃 데이터의 4 개 특성을 PCA 로 2 개 주성분으로 압축하면 ?

PC2 ↑



PC1 →

● setosa

● versicolor

● virginica

PC1 + PC2 = 95.81%

4 차원을 2 차원으로 줄였는데도 정보의 96% 가 그대로 유지됩니다 .

게다가 2D 로 그려보니 세 품종이 뚜렷하게 구분됩니다 — 압축했는데 오히려 더 잘 보입니다 !

실제 결과 미리보기 — 성인 소득 (26 개 특성)



13 강의 `get_dummies()` 결과 (26 개 특성) 에 PCA 를 적용하면 어떨까요 ?

70% 분산 → 14 개 필요

80% 분산 → 16 개 필요

90% 분산 → 19 개 필요

95% 분산 → 21 개 필요

26 개를 21 개로 ... 압축 효율이 Iris 만큼 좋지는 않습니다 !

왜 이렇게 차이가 날까요 ? Iris 의 4 개 특성은 서로 상관관계가 높습니다 (꽃이 크면 대부분의 부위가 함께 커짐). 반면 `get_dummies()` 로 만든 26 개의 원 - 핫 특성들은 서로 거의 독립적입니다 (한 직업이면 다른 직업은 자동으로 0). 압축할 ' 중복 정보 ' 가 애초에 많지 않다는 뜻입니다 . 실습에서 직접 확인해봅시다 .

PCA, 코드로는 이렇게 간단합니다

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1) 표준화
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2) PCA 적용 (2 개 주성분으로 압축 )
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 3) 설명된 분산 비율 확인
print(pca.explained_variance_ratio_)
```

오늘의 핵심 요약



특성이 많아질수록 계산 · 시각화 · 과적합 문제 (차원의 저주) 가 생깁니다 .



PCA 는 정보 (분산) 를 최대한 보존하며 더 적은 축으로 데이터를 압축합니다 .



PCA 전에는 반드시 표준화 (StandardScaler) 를 거쳐야 합니다 .



설명된 분산 비율과 누적 분산으로 `n_components` 를 정합니다 .



압축 효율은 데이터의 상관관계 구조에 따라 다릅니다 — 항상 잘 압축되는 건 아닙니다 .

다음은 실습입니다 Iris 로 PCA 의 ' 성공 사례 ' 를 직접 확인하고 , 성인 소득 데이터로 PCA 가 항상 만능은 아니라는 현실적인 트레이드오프까지 체험해봅니다 .

PYTHON 을 이용한 데이터 분석 · 14 강 실습

파이썬으로 PCA 차원 축소 직접 해보기

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
Iris: 4 → 2 차원 (95.8%↑)
```

실습편 (2 시간) — Iris(성공 사례) 와 성인 소득(현실 트레이드오프) 으로 PCA 를 깊이 있게 탐구

ROADMAP

2 시간의 여정을 한눈에

Part 0	환경 설정 (8 분)	라이브러리 import, Iris· 성인 소득 데이터 로드
Part 1~2	Iris 탐색 + 표준화 (18 분)	구조 복습 , StandardScaler 로 스케일 맞추기
Part 3~4	PCA 적용 + 시각화 (23 분)	설명된 분산 비율 , 스크리플롯 , 2D 산점도 , 주성분 해석
Part 4c	KNN 정확도 비교 (8 분)	원본 4 특성 vs PCA 2 성분 — 놀라운 결과 확인
Part 5~6	성인 소득 데이터 PCA (26 분)	26 개 특성 표준화 + PCA, 누적분산 그래프 , n_components 결정
Part 7	실전 트레이드오프 (24 분)	PCA 압축 데이터로 RF 재학습 , 정확도 · 속도 비교 , 원인 분석
도전 1~5	심화 문제 (35 분 , 선택)	n_components 실험 , 3D 시각화 , 주성분 해석 , Titanic PCA, 미니 프로젝트



환경 설정 체크리스트

1

Import 확인

```
sklearn.preprocessing.StandardScaler / sklearn.decomposition.PCA
```

2

Iris 로드

```
from sklearn.datasets import load_iris
```

3

성인 소득 로드

```
income_df = pd.read_csv('adult_income.csv') (13 강과 동일 파일)
```

4

구조 확인

```
iris.data.shape / income_df.shape
```

5

13 강 복습

get_dummies() 로 26 개 특성 만들었던 과정 기억나는지 확인

라이브러리 импорт 및 데이터 로드

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier

iris = load_iris()
income_df = pd.read_csv('adult_income.csv')
print('✓ 준비 완료!')
```

Iris 데이터 복습 — 4 개의 연속형 특성



12 강에서 다뤘던 붓꽃 데이터입니다 . 오늘은 이 4 개 특성을 PCA 로 압축해봅니다 .

```
X = iris.data    # (150, 4) - sepal/petal length·width
y = iris.target  # 0=setosa, 1=versicolor, 2=virginica

print(iris.feature_names)
print(f' 샘플 수 : {X.shape[0]}, 특성 수 : {X.shape[1]}')

# 4 개 특성은 서로 상관관계가 높습니다
# ( 꽃이 크면 꽃잎 · 꽃받침이 대체로 함께 커짐 )
```

표준화 — PCA 전 필수 단계

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(' 표준화 전 평균 / 표준편차 :', X.mean(axis=0).round(2), X.std(axis=0).round(2))
print(' 표준화 후 평균 / 표준편차 :', X_scaled.mean(axis=0).round(2), X_scaled.std(axis=0).round(2))
```

확인 포인트 표준화 후에는 모든 특성의 평균이 0에 가깝고, 표준편차가 1에 가까워집니다. 이제 모든 특성이 '동등한 목소리'를 갖게 되어, PCA가 특정 특성에 치우치지 않고 공정하게 분산을 계산할 수 있습니다.

PCA 적용하기 — 4 차원을 2 차원으로

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print(f' 원본 shape: {X_scaled.shape} → PCA 후 shape: {X_pca.shape}')
print(f' 설명된 분산 비율 : {pca.explained_variance_ratio_}')
print(f' 합계 : {pca.explained_variance_ratio_.sum():.2%}') # 약 95.81%
```

(150, 4) → (150, 2) 로 shape 이 줄어든 것을 확인하세요 . 4 개였던 열이 2 개 (PC1, PC2) 로 줄었지만 , 원본 데이터의 약 95.8% 에 해당하는 정보 (분산) 는 그대로 유지됩니다 .

스크리 플롯 — 주성분 4 개를 모두 보면 ?

```
pca_full = PCA()
pca_full.fit(X_scaled)

plt.plot(
    range(1,5),
    pca_full.explained_variance_ratio_,
    'o-')
plt.xlabel(' 주성분 번호 ')
plt.ylabel(' 설명된 분산 비율 ')
plt.title(' 스크리 플롯 ')
plt.show()
```

실제 결과

PC1: 72.96% / PC2: 22.85%
PC3: 3.67% / PC4: 0.52%

PC1 에서 PC2 까지는 큰 폭으로 떨어지지만 , PC2 에서 PC3 부터는 급격히 완만해집니다 — 이 지점이 ' 팔꿈치 ' 입니다 .

→ 2 개면 충분하다는 신호 !

2D 산점도 시각화 — 품종별로 색칠하기

```
plt.figure(figsize=(9, 6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1],
                      c=y, cmap='viridis', s=60, edgecolor='k')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%})')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%})')
plt.title('Iris - PCA 2 차원 시각화 ')
plt.legend(handles=scatter.legend_elements()[0],
           labels=list(iris.target_names))
plt.show()
```

4 차원이라 원래는 그릴 수 없었던 데이터가 , PCA 덕분에 2 차원 산점도로 한눈에 보입니다 . setosa 는 뚜렷이 분리되고 , versicolor·virginica 는 살짝 겹칩니다 .

주성분 해석하기 — PC1 과 PC2 는 무엇을 의미할까 ?

```
loadings = pd.DataFrame(pca.components_, columns=iris.feature_names,  
                        index=['PC1', 'PC2'])  
print(loadings.round(3))
```

실제 결과 해석 PC1 은 petal length(0.58), petal width(0.57), sepal length(0.52) 에 고르게 큰 가중치를 줍니다 — 꽃 전체의 ' 크기 ' 를 종합한 지표로 볼 수 있습니다 . PC2 는 거의 sepal width(0.92) 하나에 좌우됩니다 — ' 꽃받침의 폭 ' 정보를 담당합니다 .

압축해도 성능이 유지될까 ? — KNN 으로 확인

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X_scaled, y, test_size=0.3,
    random_state=42, stratify=y)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_tr, y_tr)
acc_orig = knn.score(X_te, y_te)

# PCA(2) 로 압축한 데이터로도 학습
pca2 = PCA(n_components=2)
X_tr_p = pca2.fit_transform(X_tr)
```

실제 결과 — 놀랍습니다 !

원본 4 개 특성 : 91.11%

PCA 2 개 성분 : 91.11%

특성을 절반으로 줄였는데 정확도는 완전히 동일합니다 ! Iris
는 PCA 로 압축해도 정보 손실이 실질적으로 없다는
뜻입니다 .

이제 성인 소득 데이터로 — 26 개 특성 복습



13 강에서처럼 `get_dummies()` 로 원 - 핫 인코딩하면 26 개 특성이 됩니다 . 이번엔 PCA 로 압축해봅니다 .

```
X_inc = pd.get_dummies(income_df.drop(['id', 'income'], axis=1),
                        drop_first=True)
y_inc = (income_df['income'] == '>50K').astype(int)

X_tr2, X_te2, y_tr2, y_te2 = train_test_split(
    X_inc, y_inc, test_size=0.3, random_state=42)

print(f' 특성 개수 : {X_inc.shape[1]} 개 ') # 26 개
```

표준화 + PCA 전체 적용

```
scaler2 = StandardScaler()
X_tr2_s = scaler2.fit_transform(X_tr2)
X_te2_s = scaler2.transform(X_te2)

pca_inc = PCA() # n_components 지정 안 하면 전체 26 개 축 모두 계산
pca_inc.fit(X_tr2_s)

print(pca_inc.explained_variance_ratio_[:5].round(4))
# [0.0826 0.0679 0.0639 0.0539 0.0494] - PC1 이 겨우 8.26%!
```

누적 분산 그래프로 비교하기

```
cum_var = np.cumsum(pca_inc.explained_variance_ratio_)
plt.plot(range(1, 27), cum_var, 'o-')
plt.axhline(0.9, color='red', linestyle='--') # 90% 기준선
plt.xlabel('주성분 개수'); plt.ylabel('누적 설명된 분산'); plt.show()
```

Iris 와 비교해보세요 Iris 는 그래프가 초반에 가파르게 치솟았지만 (2 개 만에 96%), 성인 소득 데이터의 그래프는 거의 일직선에 가깝게 완만히 올라갑니다 . 이는 26 개 특성이 서로 중복 정보가 적어서 , 어느 하나도 압도적으로 중요하지 않다는 뜻입니다 .

PART 6

몇 개의 주성분이 필요할까 ?

```
for target in [0.7, 0.8, 0.9, 0.95]:  
    n = np.argmax(cum_var >= target) + 1  
    print(f'{target:.0%} 분산 → n_components={n}')
```

실제 결과표

70% 분산 → 14 개 80% 분산 → 16 개 90% 분산 → 19 개 95% 분산 → 21 개

(전체 26 개 중 21 개나 있어야 95% 를 설명합니다 — Iris 는 4 개 중 2 개로 충분했던 것과 대조적입니다 .)

PCA 압축 데이터로 RF 재학습 — 실전 트레이드오프

```
pca19 = PCA(n_components=19)    # 90% 분산 유지
X_tr2_p = pca19.fit_transform(X_tr2_s)
X_te2_p = pca19.transform(X_te2_s)

rf_pca = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
rf_pca.fit(X_tr2_p, y_tr2)
```

실제 결과 — 예상과 다릅니다 원본 26 개 특성 RF: 정확도 76.67%, 학습 시간 약 0.35 초 . PCA 19 개 성분 RF: 정확도 72.67%(4.0%p 하락), 학습 시간 약 0.55 초 (오히려 더 느림). 압축이 항상 정확도와 속도를 개선하지는 않습니다 !

왜 이번엔 PCA가 도움이 안 됐을까 ?



원 - 핫 인코딩 특성은 서로 독립적

occupation 이 'Exec-managerial' 이면 다른 occupation 컬럼은 자동으로 0 입니다 . 애초에 압축할 중복 정보가 적습니다 .



트리 기반 모델은 원본 특성과 궁합이 좋음

Random Forest 는 ' 이 컬럼이 0 인가 1 인가 ' 처럼 명확한 기준으로 가지를 나눌 때 잘 작동합니다 . PCA 로 섞인 연속값은 오히려 트리가 해석하기 더 어려울 수 있습니다 .



작은 데이터에서는 속도 이점도 적음

PCA 의 속도 이점은 특성이 수백 ~ 수천 개이거나 데이터가 매우 클 때 두드러집니다 . 오늘처럼 26 개 , 1,000 행 규모에서는 오히려 변환 자체의 오버헤드가 더 클 수 있습니다 .

그래도 시각화는 해볼 수 있습니다

```
pca2_inc = PCA(n_components=2)
X_2d = pca2_inc.fit_transform(X_tr2_s)
plt.scatter(X_2d[:,0], X_2d[:,1], c=y_tr2, cmap='coolwarm', alpha=0.5)
plt.show() # PC1+PC2 는 겨우 15.05%의 분산만 설명
```

Iris 와 비교해보세요 Iris 는 2D 그림에서 품종이 뚜렷하게 갈렸지만 , 이번엔 $\leq 50K$ 와 $> 50K$ 그룹이 뒤섞여 흐릿하게 겹칠 것입니다 . PC1+PC2 가 전체 분산의 15% 밖에 설명하지 못하니 당연한 결과입니다 . '2 차원으로 그려봤다' 는 것과 '그 그림이 의미 있다' 는 것은 다른 이야기라는 점을 기억하세요 .

CHALLENGE 1

n_components 을 바꿔가며 정확도 실험 (선택)

TODO: 성인 소득 데이터에서 n_components 를 2, 5, 10, 15, 19, 21, 26 로 바꿔가며 RF 정확도가 어떻게 변하는지 그래프로 그려보세요 .

```
for n in [2, 5, 10, 15, 19, 21, 26]:  
    # PCA 적용 후 RF 학습 , 정확도 기록  
    ...
```

실제 결과 : 2 개 (62.67%) → 5 개 (68.33%) → 10 개 (71.67%) → 15 개 (73.00%) → 19 개 (72.67%) → 21 개 (75.00%) → 26 개 (76.67%). 대체로 늘어날수록 좋아지지만 항상 단조롭지는 않습니다 .

CHALLENGE 2

3 차원으로 시각화하기 (선택)

TODO: Iris 데이터를 PC1, PC2, PC3(3 차원) 로 압축해서 3D 산점도를 그려보세요 .

```
from mpl_toolkits.mplot3d import Axes3D
pca3 = PCA(n_components=3)
X_3d = pca3.fit_transform(X_scaled)

fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X_3d[:,0], X_3d[:,1], X_3d[:,2], c=y, cmap='viridis')
plt.show() # PC1+PC2+PC3 = 99.48% 분산 !
```

CHALLENGE 3

주성분 해석 연습 (선택)

TODO: 성인 소득 데이터 PCA 의 첫 번째 주성분 (PC1) 에서 가중치 (loading) 의 절댓값이 가장 큰 원래 특성 5 개를 찾아보세요 . 그 특성들이 어떤 공통점을 가지는지 설명해보세요 .

```
pc1_loadings = pd.Series(pca_inc.components_[0], index=X_inc.columns)
print(pc1_loadings.abs().sort_values(ascending=False).head(5))
```

CHALLENGE 4

Titanic 에도 PCA 를 적용해보면 ? (선택)



Pclass, Sex, Age, Fare 4 개 특성에 PCA 를 적용하고 결과를 Iris 와 비교해보세요 .

```
X_tt = titanic[['Pclass', 'Sex', 'Age', 'Fare']] # Sex 는 0/1 인코딩 필요
X_tt_s = StandardScaler().fit_transform(X_tt)
pca_tt = PCA().fit(X_tt_s)
print(pca_tt.explained_variance_ratio_)
```

실제 결과 : [0.2688, 0.2510, 0.2435, 0.2367] — 4 개 성분이 거의 균등합니다 ! PC1+PC2 도 51.98% 뿐입니다 . Titanic 의 4 개 특성도 서로 상관관계가 낮다는 뜻으로 , 성인 소득 데이터와 비슷한 패턴입니다 .

CHALLENGE 5

미니 프로젝트 — 나만의 압축 기준 찾기 (선택)



정확도 손실을 2%p 이내로 유지하면서, 성인 소득 데이터를 최대한 압축해보세요 .

1. 원본 26 개 특성 RF 정확도 (76.67%) 를 기준으로 삼으세요 .
2. `n_components` 를 바꿔가며 RF 를 학습시키고 정확도를 확인하세요 .
3. '정확도 74.67% 이상을 유지하는 가장 작은 `n_components`' 를 찾아보세요 .
4. 몇 개까지 줄일 수 있었나요 ? 원본 대비 몇 % 압축했나요 ?



도전 1 의 결과표를 참고하면 힌트를 얻을 수 있습니다 . Challenge 4 의 답은 정해져 있지 않으니 자유롭게 탐색하세요 .

CHECKLIST

실습 완료 확인 사항



Part 0~2

환경 설정 , Iris 로드 , 표준화 (StandardScaler)



Part 3~4

PCA 적용 , 설명된 분산 비율 · 스크리플롯 , 2D 시각화 , 주성분 해석 , KNN 정확도 비교



Part 5~6

성인 소득 데이터 26 개 특성 표준화 , PCA 전체 적용 , 누적분산 그래프 , n_components 결정



Part 7

PCA 압축 데이터로 RF 재학습 , 정확도 · 속도 비교 , 압축이 잘 안 된 이유 설명



도전 1~5

n_components 실험 , 3D 시각화 , 주성분 해석 , Titanic PCA, 미니 프로젝트 (선택)

14 강 마무리



PCA 는 정보를 최대한 보존하며 특성 개수를 줄이는 방법입니다 .



Iris 처럼 특성끼리 상관관계가 높은 데이터는 PCA 가 극적으로 효과적입니다 (4→2, 정확도 그대로) .



성인 소득 데이터처럼 원 - 핫 인코딩된 독립적 특성이 많으면 PCA 효과가 제한적입니다 .



차원 축소는 만능이 아니라 ' 데이터의 상관관계 구조 ' 에 달린 도구입니다 .



다음 시간에는 오늘 배운 것을 더 확장해 봅시다 .

수고하셨습니다 ! 오늘 배운 PCA 는 데이터 전처리 도구함의 중요한 한 축입니다 .